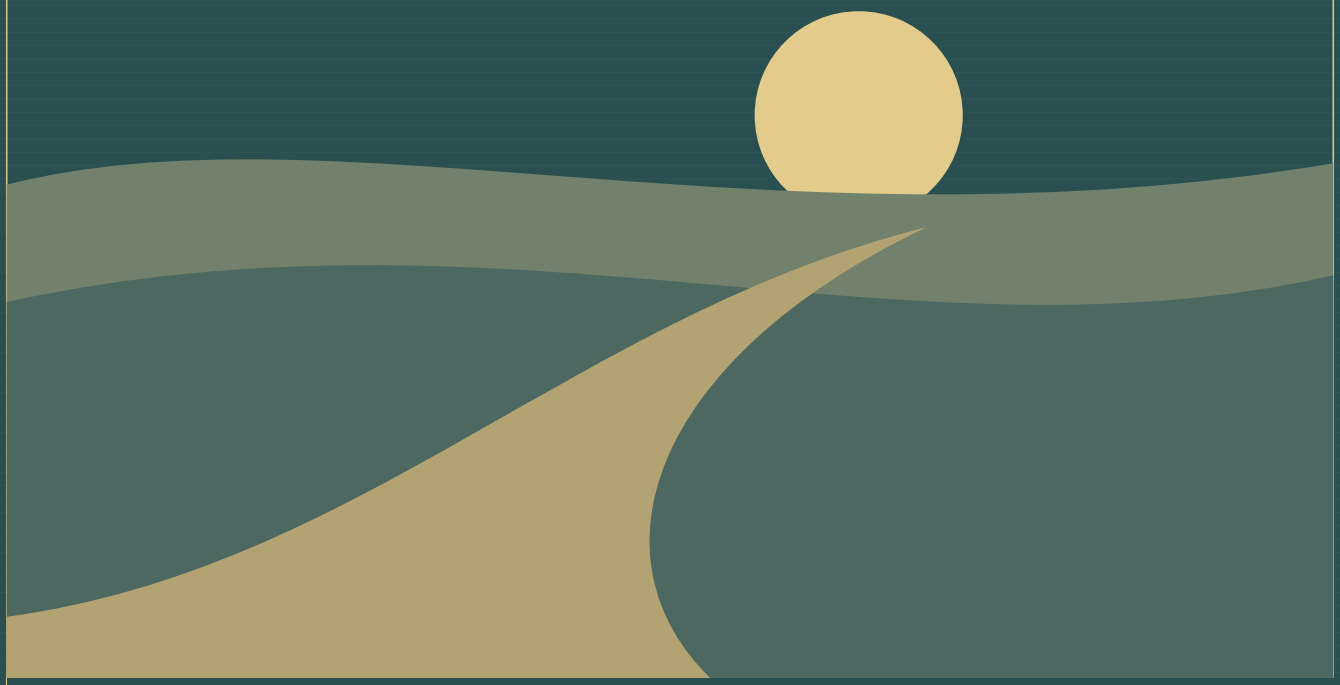
A PRACTICAL FIELD GUIDE

The LIVING REPOSITORY

Persistent AI collaboration
with Git and GitHub

READ · SHAPE · RETURN



6 LESSONS · 90 MINUTES · 1 WORKSPACE

Contents

MAP 01 · A six-file workspace . . . 3

MAP 02 · From a change to a shared repository . . . 4

MAP 03 · Choose recovery by history . . . 5

The Living Repository . . . 6

Start here . . . 8

Codex with your local repository . . . 10

01 · Orient (10 minutes) . . . 12

02 · Direct (12 minutes) . . . 13

03 · Ground (13 minutes) . . . 14

04 · Decide (13 minutes) . . . 15

05 · Inspect and recover (22 minutes) . . . 17

06 · Final challenge (20 minutes) . . . 18

Final challenge: resume, change, and recover . . . 20

Final challenge rubric . . . 22

Evidence worksheet . . . 23

Copy-ready Codex prompts . . . 24

Git you can see . . . 26

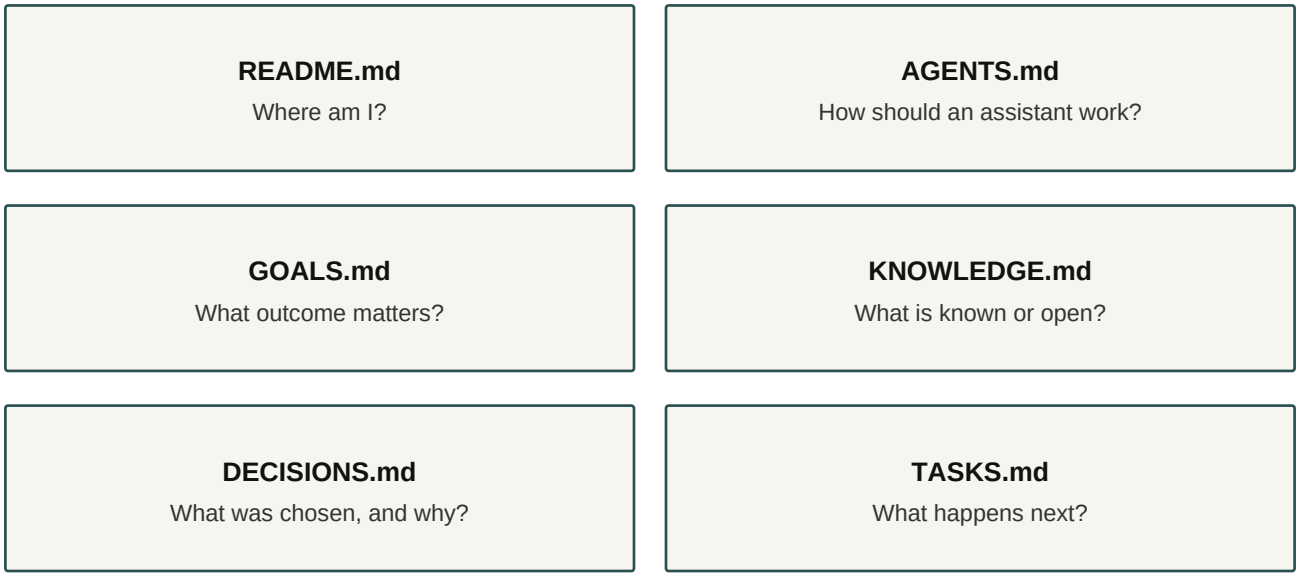
The Living Repository — desk reference . . . 28

Course pack and repository setup . . . 29

Optional browser adaptation . . . 30

MAP 01 · A six-file workspace

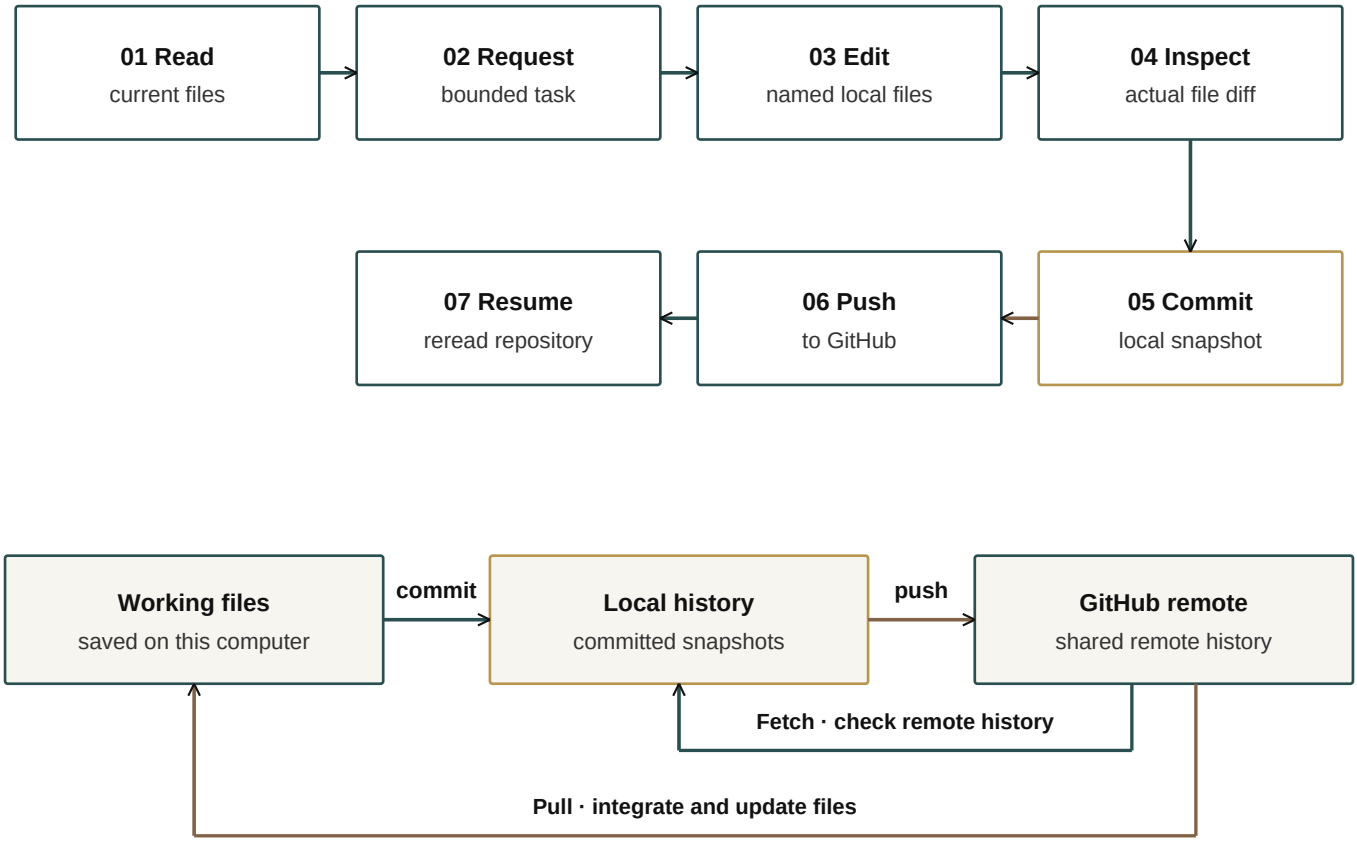
One role per file makes context easier to find and maintain.



In words. README is the index; AGENTS gives collaboration rules; GOALS names outcomes; KNOWLEDGE separates facts from uncertainty; DECISIONS records choices and reasons; TASKS tracks next steps and blockers.

MAP 02 · From a change to a shared repository

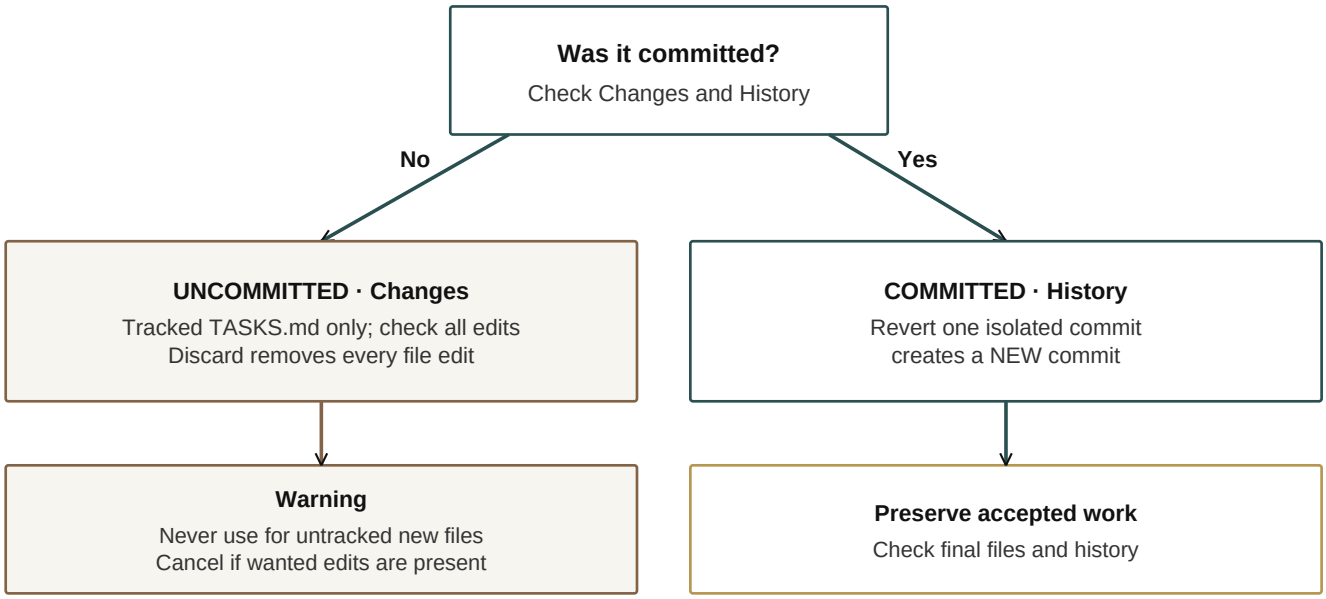
Saving, committing and pushing happen at different points.



In words. Codex reads the repository files and makes bounded local edits. The learner inspects the actual file diff before committing. A commit records a local snapshot. Push sends commits to GitHub. Fetch updates remote-tracking information; pull integrates remote work into the current branch and updates working files.

MAP 03 · Choose recovery by history

Check the Changes view and History before acting.



In words. For an unwanted uncommitted edit, use discard only on a tracked existing TASKS.md after confirming it contains no wanted edits: discard removes all uncommitted edits in that file, and it is not for untracked new files. Cancel if wanted edits are present. For an unwanted committed change, rehearse against an isolated commit; revert creates a new commit reversing it while preserving accepted history. Check final files and history.

The Living Repository

A practical introduction to persistent AI collaboration using Git and GitHub

6 lessons · 90 minutes · 1 reusable workspace

Learn to work with Codex in VS Code while keeping project context in six ordinary files. You will orient yourself in the workspace, give Codex clear bounded tasks, separate facts from assumptions, record a changed decision, inspect actual file changes, and recover from mistakes in GitHub Desktop.

The running example is a fictional community skill-share for 12 attendees. Its starting budget is 60 fictional units, then changes to 40; attendance stays fixed. The final challenge changes the budget to 30. The main path uses GitHub Desktop, VS Code with the official OpenAI Codex extension, a supported ChatGPT sign-in, and a private GitHub repository. No programming, command line, or API key is required. See Start here for setup and the local Codex setup guide.

Begin: Set up your workspace → Take Lesson 1 → Complete the challenge

Course map

Complete the setup before the timed lessons. The six lessons total exactly 90 minutes.

Lesson	Focus	Time
01 · Orient	Understand the six files and seed the scenario	10 min
02 · Direct	Give Codex bounded, useful instructions	12 min
03 · Ground	Separate facts, assumptions, and unknowns	13 min
04 · Decide	Record the 60-to-40 budget change consistently	13 min
05 · Inspect and recover	Review diffs and recover from two kinds of mistake	22 min
06 · Final challenge	Resume, change, inspect, push, and recover independently	20 min

Each lesson has five parts: **Problem, Concept, Demonstration, Exercise, Checkpoint**. Codex tasks use the opened local repository. Codex can make the requested local file edits; you inspect the actual diff in GitHub Desktop and decide what to commit and push.

Materials

- Start here: prerequisites, setup, and the supported local workflow.
- Codex local setup: setup notes for VS Code and Codex.
- Browser AI fallback: optional text-only alternative if you cannot use local Codex.
- Prompt collection: reusable task wording.
- Three visual maps: file roles, local/remote Git, and recovery.
- Visual Git guide: save, commit, push, and recovery in GitHub Desktop.
- Final challenge and rubric: independent task and evidence criteria.
- Starter distribution instructions: how to obtain and use the six-file starter package.
- Assembled guide: all six lessons in one continuous reading copy.

Your practice workspace

The repository has exactly these six Markdown files:

File	Holds
`README.md`	A short project overview and links to the other files

File	Holds
`AGENTS.md`	Persistent collaboration guidance for Codex
`GOALS.md`	Outcomes the project is trying to achieve
`KNOWLEDGE.md`	Facts, assumptions, and questions, with uncertainty labeled
`DECISIONS.md`	Choices made, their reasons, and what would change them
`TASKS.md`	Current next actions and their status

Do not add real personal or sensitive information to this practice repository. Ask Codex to read the named files before relying on them. Review any edits in GitHub Desktop before committing; an AI reply or a saved edit is not itself a commit or push.

What completion looks like

In a fresh Codex chat opened on the same local repository, you can ask it to reread the files and report current facts and unknowns. You can then change the budget from 40 to 30 without changing attendance, inspect and commit the accepted edits, push them to the private GitHub repository, and recover from a separate rehearsal commit without undoing accepted work. The final challenge explains the evidence to collect.

Start here

The main course path uses Codex inside VS Code to read and edit the local project files. GitHub Desktop shows what changed and handles commits and pushes. Both apps must point to the **same local repository folder**. You decide whether each proposed local edit belongs in the saved project history.

Before the 90-minute course

Allow time before class to install and sign in to GitHub Desktop, create a private GitHub repository, clone it locally, and add the starter files. Then set up VS Code and Codex. These steps are outside the six timed lessons.

1. On a Windows or macOS computer, install GitHub Desktop and sign in to your GitHub account.
2. On GitHub, create a new repository named something like `living-skill-share`. Set it to **Private** and leave the options to add a README, `.gitignore`, or license unchecked so it is empty.
3. In GitHub Desktop, clone that repository to a folder you can find again.
4. Use the course's distribution instructions to get the starter package. Copy only the six Markdown files and hidden `.gitignore` into the cloned repository, not the package's enclosing folder.
5. In GitHub Desktop, review the file list, commit with the summary `Add six-file skill-share starter`, and push. Confirm the six Markdown files appear in the private GitHub repository and GitHub Desktop reports the repository is up to date.
6. Install Visual Studio Code and the official OpenAI Codex extension. Sign in to the extension with a ChatGPT account that Codex supports. Follow the current Codex IDE setup guide and official extension documentation if the installation flow differs from these notes.
7. Open the cloned learner repository folder in VS Code, not the course source or worked example. Confirm it is the exact local folder GitHub Desktop has open; use the repository path shown in GitHub Desktop to compare. Start a **Local task** in Codex for this folder. Do not choose Cloud or a worktree for this course.
8. Ask Codex to read `AGENTS.md` and the five other Markdown files, then report their filenames. Do not ask it to edit yet. Confirm the files are available in the opened local project.

The starter package supplies files; it does not create a repository or publish anything for you. This course does not provide a hosted template URL. The six lesson files, not the setup steps above, make up the 90-minute course.

What you need

- A Windows or macOS computer with GitHub Desktop and VS Code installed.
- A GitHub account and a private repository you control.
- The official OpenAI Codex extension in VS Code and a supported ChatGPT sign-in.
- The six-file starter package from the local course distribution.

No programming, command line, API key, or special project tooling is required. Codex can edit the local files after you ask it to; GitHub Desktop is where you inspect the resulting changes and decide what to commit and push. Account and product availability can vary. If local Codex is unavailable, the optional browser AI fallback explains the alternative; the main lessons use local repository access.

How the local workflow works

Start Codex from VS Code with the cloned project folder open. For each task, name the exact files to read, ask for a read-only report first when facts matter, and name the only files it may edit. Codex reads and writes files in that local workspace according to the task and tool access you give it. Never assume it has read a file merely because it is in your repository: ask it to name the files it used, and check its claims against the files.

After Codex edits a file, save is handled by the editor/extension. Switch to GitHub Desktop, select the same repository, and inspect the actual changed-file list and line-by-line diff. If the changes are right, commit them. Push to send that commit to GitHub. Codex's local edit does not commit or push the change for you.

Start a fresh Codex conversation for the final challenge while keeping the same local repository folder open. Ask it to reread the files and report their current state. This checks whether the saved workspace carries the context into a new conversation; do not ask it to remember the previous chat.

`AGENTS.md` gives persistent guidance about how Codex should work with the project files. It does not grant permission to spend money, send messages, book a venue, or publish information, and it does not replace your review of Codex's actions. Keep the fictional practice details in the repository and decide what changes to accept.

See the course map, the Codex local setup guide, and the GitHub Desktop guide before starting Lesson 1.

Codex with your local repository

This course uses **Visual Studio Code (VS Code), the official OpenAI Codex extension, and GitHub Desktop** on Windows or macOS. VS Code is simply your file editor here: you will write Markdown, not programs. Setup is additional to the 90 minutes. You need an account with access to Codex; availability and usage limits depend on your account. No API key is required for the ChatGPT sign-in path.

Open one shared folder

1. Complete the repository creation and clone steps in Start here.
2. In GitHub Desktop, select your learner repository and use **Repository → Show in Finder / Show in Explorer**. Note its full folder location.
3. Install VS Code. Choose **File → Open Folder** and open that exact folder. The six workspace files must appear at the top of its Explorer. Open only your learner workspace, not the course source, a parent folder or the completed example.
4. Install the official **Codex** extension from OpenAI, using the installation link in the official extension guide. Open its sidebar and sign in with your supported ChatGPT account.
5. Keep the task local. If offered Cloud, a separate worktree, or another working folder, return to the local checkout you opened. The course uses one branch and one working copy. You do not need a cloud repository connection.
6. Review the tool's permission controls. Use read-only access for orientation where available, then allow writes within this workspace for the named exercise. Keep approval prompts enabled for operations beyond that scope. Permission labels vary by version. A prompt or AGENTS.md instruction guides the assistant; the tool's access controls enforce its actual permissions.

Local refers to the location of your working files. It does not mean the AI runs offline: the service still processes context you provide through its tools. Use the fictional course scenario while learning.

Verify reading before editing

Start a new Codex chat in that folder and send:

```
Read README.md, AGENTS.md, GOALS.md, KNOWLEDGE.md, DECISIONS.md and TASKS.md from this repository.
    Do not edit files or run Git operations. Report the workspace folder, the files you could read,
    one fact with its source filename, and any blank sections. If you cannot read a file, say so
    rather than guessing.
```

Open the cited file yourself and compare. At the blank starter stage, an empty section is the correct result. Confirm the reported folder matches GitHub Desktop's folder. Access to a folder does not prove the assistant has read every file: explicitly name the files each task needs.

Verify a bounded edit

Use Lesson 2's small README edit as the write check. Ask Codex to edit only the named file, then open GitHub Desktop's **Changes** view. The actual changed lines must appear in the same repository. A chat reply saying “done” is not evidence that the file changed. Read the diff before committing. The learner performs commits, pushes, discards and reverts in GitHub Desktop throughout this course.

If no change appears, check the full folder path, whether Codex only proposed text, write permission, unsaved editor buffers and any separate worktree. Resolve this before continuing. Do not copy files between different checkouts to conceal a mismatch.

Resume from files

Commit accepted work, then start a new Codex chat in the same folder. Ask it to read the six files again and report the latest decision and next action before editing. This tests whether the repository carries the state; an old chat summary can be stale.

Sources and scope

Product guidance checked 17 October 2026: Codex IDE extension for installation, local work and reviewing changes; AGENTS.md instructions for persistent project guidance. The particular six-file layout, task boundaries and learner-controlled Git steps are course choices. Interface labels and account access may change; follow the current official installation instructions if your screen differs. This release has not been tested end to end in the desktop applications with a beginner.

Return to setup · Optional browser adaptation

01 · Orient (10 minutes)

01 · Problem · 1 minute

Six blank starter files are ready, but the project facts have not been entered yet. An assistant can sound confident about details it has not read. The fictional community skill-share gives us a harmless project to organize: it is planned for 12 attendees, with a starting budget of 60 fictional units.

02 · Concept · 2 minutes

Treat the repository as a six-page project notebook. `README.md` is the index; `AGENTS.md` is persistent guidance for Codex; `GOALS.md` records the intended outcome; `KNOWLEDGE.md` holds facts and uncertainty; `DECISIONS.md` records accepted choices; `TASKS.md` tracks next actions. A fresh chat does not carry reliable memory of the previous conversation. The local files preserve context, and Codex can reread them when you ask.

03 · Demonstration · 2 minutes

Open the cloned repository in VS Code and start a local Codex task. Ask for a read-only orientation before adding the scenario:

> *Read* `AGENTS.md`, `README.md`, `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, *and* `TASKS.md` *from this open local repository. Do not change any file. First list the filenames you successfully read and say what each is for. Then tell me whether these files currently state attendance, starting budget, date, or venue. Treat missing information as unknown; do not use details from this prompt as if they were already in the files.*

A plausible reply names the six files and says the starter does not yet record attendance, budget, date, or venue. Confirm the answer by opening the files in VS Code. If Codex claims the files contain facts that are not there, ask it to show which file and passage supports the claim.

04 · Exercise · 4 minutes

Now ask Codex to seed the fictional project. It should edit the local files directly:

> *Read all six project files first. Update only* `GOALS.md` *and* `KNOWLEDGE.md`*. In* `GOALS.md`*, record the goal of planning a fictional community skill-share for 12 attendees. State that the plan should fit the initial 60-unit budget, attendance stays at 12, and the exercise gives no permission to spend. In* `KNOWLEDGE.md`*, list as confirmed facts that the attendance target is 12 and the starting budget is 60 fictional units. Label the event date and venue as unknown, and say all scenario details are fictional. Preserve the existing headings and other useful text. Do not change other files. After saving the edits, report which files you changed and summarize the facts you recorded.*

Open `GOALS.md` and `KNOWLEDGE.md` in VS Code and check the wording. Start a fresh Codex chat in the same open repository and ask: “Read the six files again. Which files state the attendance and starting budget? What are the date and venue? Name the files you used; make no edits.” A plausible read-back reports 12 and 60 from `GOALS.md` and `KNOWLEDGE.md`, with date and venue unknown. Correct any mismatch before continuing.

05 · Checkpoint · 1 minute

The fresh chat read the saved local files rather than relying on the first chat. In GitHub Desktop, select the same repository folder, inspect the actual diff, and confirm only `GOALS.md` and `KNOWLEDGE.md` changed. Commit them as `Seed fictional skill-share context`, then push. Your initial starter commit remains separate. Continue to 02 · Direct.

02 · Direct (12 minutes)

01 · Problem · 2 minutes

“Help with my event” does not say what Codex should read, which files it may edit, or what result you want. A broad request can lead to invented plans or changes that are hard to inspect.

02 · Concept · 2 minutes

Give Codex a named set of source files, one concrete task, clear limits, and a report request. `AGENTS.md` is persistent project guidance Codex should read; a task prompt describes the current work. Those instructions help the assistant work with the files, but they do not grant permission to spend money, send messages, book a venue, or publish anything. You decide whether local edits are acceptable, and GitHub Desktop gives you a visible diff before you commit.

03 · Demonstration · 3 minutes

In the local Codex task for the open repository, ask for a read-only check first:

> *Read* `AGENTS.md` *and the five other Markdown project files from this local repository. Do not edit. List the filenames you read. In three bullets, summarize (1) how you should use the project files, (2) how you should handle unknown details, and (3) the current next action. Explain that project guidance does not authorize spending, messages, bookings, or publishing. Do not assume you have read any file you cannot name.*

A plausible reply says it should consult the files and preserve known facts; identify unknown details rather than invent them; and look to `TASKS.md` for the next action. It also says the repository gives no permission for real-world actions. Confirm those points against the files. Now ask Codex for one bounded local edit:

> *Rewrite the overview in the local* `README.md` *as two clear sentences. Use the existing project files as your source. Do not repeat numeric attendance or budget values; point readers to* `GOALS.md` *and* `KNOWLEDGE.md` *for current project facts. Do not invent a date, venue, or event details. Preserve the other five files. Save the local edit and report exactly what you changed.*

A plausible result describes the fictional skill-share workspace and points readers to its goals and known facts, without duplicating numbers. Check that Codex reports only `README.md`.

04 · Exercise · 4 minutes

Open `README.md` in VS Code and compare it with Codex's report. If it repeats numeric attendance or budget values, invents an event detail, or changes another file, tell Codex to correct only `README.md` and inspect again. In GitHub Desktop, select the same repository and review the changed-file list and actual line-by-line diff. Confirm the diff is limited to the overview you asked for. Commit with `Clarify skill-share overview` and push. For the difference between saving, committing, and pushing, use the visual Git guide.

05 · Checkpoint · 1 minute

Codex read the supplied project guidance, edited only the named local file, and reported the edit. You verified that claim against the actual GitHub Desktop diff before committing and pushing. Continue to 03 · Ground.

03 · Ground (13 minutes)

01 · Problem · 2 minutes

Project notes often mix confirmed details with guesses. If Codex turns “we might use a library” into “the library is booked,” the workspace becomes misleading. A helpful local assistant should show which file supports a fact and say when a detail is missing.

02 · Concept · 2 minutes

Keep facts, assumptions, and unknowns distinct in `KNOWLEDGE.md`. A fact is supported by the current scenario or another named source; an assumption is a temporary belief that needs checking; an unknown has no answer yet. `DECISIONS.md` is for choices you accept, and `TASKS.md` is for actions. A suggestion from Codex does not become fact or a decision until you decide to record it.

03 · Demonstration · 3 minutes

Start a fresh Codex chat in VS Code with the same local repository open. Ask it to reread the files; do not refer to the previous chat:

> *Read* `AGENTS.md`, `README.md`, `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, *and* `TASKS.md` *from this local repository. Do not edit anything. First report which files you could read. Then make a three-column table labeled Fact, Assumption, and Open question. Include only information supported by the files, and name the source filename for each row. Check whether the files confirm attendance of 12 and a starting budget of 60 units. Do not guess a date, venue, or later budget.*

A plausible reply identifies 12 attendees and a 60-unit starting budget as facts supported by `GOALS.md` or `KNOWLEDGE.md`, and says the date and venue are unknown. If Codex claims a missing detail is confirmed, ask which file states it and correct the record if no such source exists.

04 · Exercise · 5 minutes

Ask Codex to make source and uncertainty labels clearer in `KNOWLEDGE.md` only:

> *Read the six project files. Update only* `KNOWLEDGE.md`*. Keep the confirmed attendance target of 12 and starting budget of 60 fictional units. Mark both facts as supplied by the course scenario. Keep the event date and venue under Unknowns. Keep assumptions separate from facts, and do not add an assumption or event detail that the files do not support. Preserve the existing useful headings and all six-file roles. Save the local edit and report the filename and the labels you used.*

Check `KNOWLEDGE.md` in VS Code against the scenario. In GitHub Desktop, inspect the actual diff and confirm only the intended labels changed. If Codex changed another file or introduced an unsupported claim, ask it to correct that specific local file before committing. Commit a sound edit as `Label skill-share facts and questions`. If the existing file already meets the request and no edit is needed, do not create a pointless commit; confirm the read-only check instead.

05 · Checkpoint · 1 minute

Codex read the repository in the fresh chat and named file sources for its summary. `KNOWLEDGE.md` separates confirmed scenario facts from unknown date and venue; any edit is visible in GitHub Desktop and limited to the requested file. Continue to 04 · Decide.

04 · Decide (13 minutes)

01 · Problem · 2 minutes

The project changes: the planning budget is now 40 units, while attendance remains 12. Updating only one file could leave another file reporting the old cap. The record needs to show what changed and which value is current.

02 · Concept · 2 minutes

Before an edit, ask Codex to read all six files and map repeated values. Then use a bounded task to update the files that carry current project state: `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, and `TASKS.md`. Keep 60 as superseded history only in `DECISIONS.md`; `KNOWLEDGE.md` should say 40 is current. If a different file repeats a current budget, update it or remove the duplicate. Ask Codex to save its local edits, then use GitHub Desktop to review the actual changed-file list and diff.

03 · Demonstration · 3 minutes

Ask Codex for a read-only impact map:

> *Read all six project files from this local repository. Do not edit. List the filenames you read, then list every place where the current budget, starting budget, superseded budget, or attendance appears. Say which files must change to record a current budget of 40 units and fixed attendance of 12. Do not invent a reason for the budget revision. Do not rely on your earlier chat; report only what the files say.*

A plausible reply distinguishes the file state from the new instruction: the files currently record 60 as the budget, while the requested change makes 40 current and moves 60 to superseded history in `DECISIONS.md`. It identifies `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, and `TASKS.md` as needing updates; if `README.md` or `AGENTS.md` repeats a current budget, that specific file needs correction too. Attendance remains 12. Check the map against the local files. If Codex overlooks a duplicate, name that file and ask it to include it in the map.

04 · Exercise · 5 minutes

Now give Codex a bounded local edit task:

> *Read all six project files. Edit* `GOALS.md`*,* `KNOWLEDGE.md`*,* `DECISIONS.md`*, and* `TASKS.md` *as needed so each shows 40 as the current cap. Keep 60 only as superseded history in* `DECISIONS.md`*;* `KNOWLEDGE.md` *should state 40 as current and not repeat 60 as history. The supplied scenario says the budget was revised but gives no further rationale; record that without inventing a cause. Inspect* `README.md` *and* `AGENTS.md`*; edit either only if it repeats a current budget value, and then correct that duplicate to 40. Do not edit any other file. Preserve attendance, the fictional nature of the example, the unconfirmed venue, and the rule that no spending or messages are authorized. Save the local edits and report each changed filename and its purpose. Do not commit or push.*

In VS Code, inspect the edited files and compare each value with the request. In GitHub Desktop, confirm the selected repository is the same folder and inspect every actual diff. Make sure the four current-state files say 40; `DECISIONS.md` keeps 60 only as superseded history; `KNOWLEDGE.md` says 40 is current; attendance remains 12; and no unsupported reason or venue claim was added. Correct any issue with Codex in a narrowly named follow-up, then inspect the new diff. Commit accepted changes as `Record revised 40-unit budget` and push.

05 · Checkpoint · 1 minute

The four current-state files agree on the 40-unit cap, with 60 preserved only in `DECISIONS.md` as superseded history. The accepted Codex edits are visible in GitHub Desktop, committed, and pushed. Continue to 05 · Inspect and recover.

05 · Inspect and recover (22 minutes)

01 · Problem · 3 minutes

Codex may make a wrong edit even when its report sounds right. You need to inspect actual file changes. Recovery also depends on whether the unwanted change is still uncommitted or already has its own commit.

02 · Concept · 4 minutes

Codex edits files in the local repository; GitHub Desktop shows saved changes. A **commit** records selected changes in local history. A **push** sends local commits to GitHub. For an unwanted uncommitted line, GitHub Desktop can discard the file's uncommitted changes. That discards **all uncommitted edits in that file**, so first confirm the file contains only the drill line; if it has wanted work, cancel and remove only the drill line with Codex. For a committed mistake, use **Revert changes in commit** on the specific isolated rehearsal commit. Desktop creates a new commit reversing it. Read the text steps in the visual Git guide.

03 · Demonstration · 4 minutes

The visual Git guide demonstrates both cases using `TASKS.md`. Follow the first example on paper: Codex adds one temporary task line; Desktop shows it as uncommitted; you discard only after confirming there is no other uncommitted edit in that file. Then follow the committed example: Codex adds one separate rehearsal line; you commit and push only that line; in History you select that exact commit and revert it. The revert appears as a new History entry, and the current task file no longer contains the temporary line. No accepted project work is part of the rehearsal commit.

04 · Exercise · 8 minutes

First make sure your accepted project changes are committed and pushed. In GitHub Desktop select your learner workspace repository (not the course source or worked example) and confirm Changes is clean. In VS Code, ask Codex:

> *Read* `TASKS.md`*. Make one local change to that file only: add this exact standalone line at the end:* `Recovery rehearsal — remove me`*. Do not change any other text or file. Do not commit or push. Report the filename changed.*

In GitHub Desktop, confirm only `TASKS.md` changed and inspect the diff. If any other local edit is present in `TASKS.md`, cancel discard and ask Codex to remove only the unwanted rehearsal line. Once the file contains only this drill line as its uncommitted change, choose **Discard changes** for `TASKS.md` and confirm. Reopen the file and verify the line is gone.

Now ask Codex to add the same isolated line again, with the same limits. Inspect the `TASKS.md` diff, commit only that line as `Add recovery rehearsal`, and push. In GitHub Desktop History, select that exact rehearsal commit and verify its only changed file and line. Choose **Revert changes in commit**. Confirm the line is absent from the current `TASKS.md` and the new recovery entry is visible in History; Desktop normally creates the revert commit directly. Push the recovery commit and verify both rehearsal and recovery entries on GitHub.

If you selected a different commit, cancel before confirming. If you already reverted the wrong commit, stop and inspect the files and History with an instructor or support person; do not make another blind reversal.

05 · Checkpoint · 3 minutes

You can explain save, commit, and push. The first uncommitted line disappeared without a commit. History shows an isolated `TASKS.md` rehearsal commit and its later recovery commit; the temporary line is absent, accepted project work remains, and both commits are visible on GitHub. Continue to 06 · Final challenge.

06 · Final challenge (20 minutes)

01 · Problem · 2 minutes

The previous Codex chat is gone. Can a fresh local session use the saved repository to resume the skill-share project, change its budget, preserve attendance, and recover from a separate mistake? This is your independent check of the full workflow.

02 · Concept · 3 minutes

Open a new Codex conversation in VS Code while keeping the same local repository folder open. Ask Codex to reread the six files and name what it could access; do not attach, paste, or rely on the earlier chat. Then request edits to the named local files. Codex saves the local changes, but GitHub Desktop is where you inspect the actual diffs, commit, and push. The final challenge sheet gives the full scenario and evidence criteria.

03 · Demonstration · 3 minutes

In the fresh Codex chat, start with a read-only task:

> *This is a fresh Codex conversation. Read* `AGENTS.md`, `README.md`, `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, *and* `TASKS.md` *from this open local repository. Do not edit. List the filenames you could read. Summarize the current attendance, current budget, superseded budget if recorded, and next task, naming the supporting files. Check whether another file repeats the current budget. Do not rely on prior chat context or infer missing details.*

A plausible response says 12 attendees, 40 units current, 60 superseded only in `DECISIONS.md`, and the next task recorded in `TASKS.md`. If Codex says it cannot read a file, follow its request to locate or open the correct local repository; do not replace this check by pasting file contents.

04 · Exercise · 9 minutes

Ask Codex to apply the challenge update in the open local repository:

> *The fictional planning budget changes from 40 to 30 units. Attendance remains 12. Read all six files first. Update* `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, *and* `TASKS.md` *so all four state 30 as the current planning limit. Keep 40 only as superseded history in* `DECISIONS.md`, *preserving its existing reason;* `KNOWLEDGE.md` *should state 30 as current and not repeat 40 as history. Inspect* `README.md` *for a repeated current amount and correct or remove that duplicate if needed. Read* `AGENTS.md` *but do not edit it; if its guidance contains a stale budget amount, stop and report that issue instead of changing the instructions. Do not edit any other files. Do not change attendance, invent a new reason, date, venue, or expense plan, weaken the no-spending/no-messages guidance, or treat a venue as confirmed. Save the local edits, name every changed file, and do not commit or push.*

In VS Code, check Codex's report against the actual local files. In GitHub Desktop, select the same repository and inspect every changed-file diff. Confirm the four current-state files say 30, 40 remains only as superseded history in `DECISIONS.md`, attendance is still 12, and no duplicate current amount or unsupported detail remains in `README.md`. If `AGENTS.md` contains a stale amount, pause and ask the instructor how to handle it; do not widen the edit. Ask Codex to correct only a named permitted file if needed, then inspect the new diff. Commit the accepted project update as `Record revised 30-unit budget`, push it, and open the private GitHub repository to verify the commit appears in its history.

For the separate recovery check, confirm GitHub Desktop has no pending changes. Ask Codex to add only `Recovery rehearsal — remove me` to `TASKS.md`, with no other edits and no commit or push. Inspect the one-line diff, commit only this rehearsal as `Add recovery rehearsal`, and push. In GitHub Desktop History, select that exact commit and use **Revert changes in commit**. Confirm the temporary line is gone and the accepted 30-unit project work remains. Push the recovery commit, then verify on GitHub that the project update, rehearsal, and recovery commits appear in order.

05 · Checkpoint · 3 minutes

Record evidence on the assessment sheet: the fresh-chat read-back from the local files; the four consistent 30-unit current-state files; 40 only as superseded history in `DECISIONS.md`; attendance at 12; the inspected accepted diff; its commit and verified private GitHub push; and the separate `TASKS.md` rehearsal commit with its Desktop recovery and verified push. A Codex report is not evidence by itself: use the files, actual GitHub Desktop diffs and History, and private GitHub commit history. Continue to the course map if you want to review the lesson sequence.

Final challenge: resume, change, and recover

Time: 20 minutes. **Work independently.** Use either your own completed course workspace or the worked Community Skill-Share repository. The scenario is fictional.

Challenge

See the local Codex setup guide before starting. The browser fallback is a separate practice adaptation; it cannot satisfy the local repository-awareness criterion.

In a fresh Codex chat, resume the project from the actual local repository, change the planning budget from 40 to 30 fictional units while keeping attendance at 12, inspect Codex's actual local edits in GitHub Desktop, and make the commits and pushes yourself. Then create and recover from one separate practice commit.

Steps

1. **Open one checkout in both tools.** Extract the worked-example repository ZIP or use your own completed course repository. In GitHub Desktop, open that repository. In VS Code, open the exact same local folder and start the official OpenAI Codex extension. Use a local task in this checkout, not Codex Cloud or a separate worktree. Confirm the file paths match the repository shown in GitHub Desktop. Configure or use a private GitHub remote. If this example has no remote, use GitHub Desktop's **Publish repository** and keep **Keep this code private** selected. Push the starting state before the timed change if needed. Public publishing is not required.
2. **Resume in a fresh local Codex chat.** Start a new local chat in VS Code with the local repository open, not a Cloud task or separate worktree. Do not attach or paste file contents. Ask Codex to read `README.md`, `AGENTS.md`, `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, and `TASKS.md` directly from the workspace, list the files it actually read, and summarize current budget, attendance, superseded budget, unknowns, and next task. Check the read-back against the actual files. If Codex cannot read the workspace files, resolve that before continuing; a browser chat or pasted text cannot satisfy the primary repository-awareness criterion.
3. **Request a bounded local change.** Tell Codex the fictional planning limit changes from 40 to 30 units; attendance stays at 12; no further reason for the change is supplied. Ask it to read all six files and make local edits only to `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, and `TASKS.md`, plus `README.md` only if it repeats a current amount that would be stale. All four state files must show 30 as current. Preserve 40 only as superseded history in `DECISIONS.md`, with the existing reason; do not repeat 40 as current or historical context in `KNOWLEDGE.md`. Do not invent a reason, date, venue, or expense plan. Keep the venue unconfirmed, attendance at 12, and the no-spending/no-messages boundary. Codex must not commit or push.
4. **Inspect, commit, and push the accepted project change.** In GitHub Desktop's **Changes** view, inspect the actual changed-file list and each diff. Reject or correct edits outside the request. Confirm every file agrees, then commit the accepted changes yourself with a clear message such as `Record revised 30-unit budget`. Push to the private repository. Open the repository on GitHub and verify that the project-update commit appears in its history.
5. **Make one isolated rehearsal commit.** In `TASKS.md`, add only this temporary line: `Recovery rehearsal — remove me`. Save and inspect the one-line diff in GitHub Desktop. Commit only that line as `Add recovery rehearsal`, then push it.
6. **Recover with GitHub Desktop.** In **History**, select that exact rehearsal commit and verify it changes only the temporary line in `TASKS.md`. Choose **Revert changes in commit**. Verify the later recovery commit appears, the temporary line is gone, and the accepted 30-unit update remains. Push the recovery commit. On GitHub, verify that both the rehearsal commit and the later recovery commit appear in the private repository history.

7. **Record evidence.** Complete `evidence.md` using local observations or privacy-safe, redacted screenshots. Keep evidence on your device. Redact account names, repository URLs, email addresses, and other identifying details. Never include credentials. A private repository is sufficient.

Finish condition

Codex read the six actual workspace files in a fresh local chat and made bounded edits to the local repository. The learner inspected the actual GitHub Desktop diff and made the project commit and pushes. `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, and `TASKS.md` show a current 30-unit limit and 12 attendees; 40 remains only as superseded history in `DECISIONS.md`; any duplicated current value in `README.md` is corrected or removed. GitHub Desktop and the private GitHub repository show the project update, separate rehearsal, and later recovery commits. A browser-only or pasted-file exercise may be useful practice, but it cannot pass the primary repository-awareness criterion. If local Codex access or either required push cannot be completed and verified, mark the challenge Not yet; do not publish the repository publicly to satisfy it.

Final challenge rubric

Mark every criterion **Pass** or **Not yet** using the learner's local repository, GitHub Desktop, fresh Codex read-back, and private GitHub history. All criteria are required for a pass. Keep the evidence private and redacted; a public repository is not required. Browser-only work or pasted file contents cannot pass the primary local repository-awareness criterion.

Criterion	Pass when observable evidence shows…	Result
Same checkout	VS Code with the official Codex extension and GitHub Desktop opened the same local repository folder; Codex used a local task in that checkout, not Cloud or a separate worktree.	Pass / Not yet
Primary repository awareness	In a fresh local Codex chat, Codex read the six files directly from the open local workspace and identified the files it actually read. No attachments, pasted file contents, or browser chat substituted for local access.	Pass / Not yet
Accurate handoff	Codex's read-back of the starting 40-unit budget, 12 attendees, known superseded 60-unit choice, unknowns, and next task matched the actual files.	Pass / Not yet
Bounded local edits	Codex made actual local edits only to `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, `TASKS.md`, and `README.md` only if needed to remove a duplicated stale current amount.	Pass / Not yet
Current budget aligned	All four state files show 30 units as current; `KNOWLEDGE.md` does not retain 40 as historical context.	Pass / Not yet
Decision history retained	`DECISIONS.md` keeps 40 only as superseded history and gives the existing reason. No other current-state file treats 40 as current.	Pass / Not yet
Other facts and boundaries preserved	Attendance remains 12, the venue remains unconfirmed, and no spending or messages are authorized.	Pass / Not yet
Actual diff inspected	Learner reviewed the changed-file list and actual diffs in GitHub Desktop and checked each edit against the request.	Pass / Not yet
Project commit and push verified	Learner made the project-update commit and it appears in the private GitHub repository history after push.	Pass / Not yet
Separate rehearsal pushed	Learner committed only `Recovery rehearsal — remove me` in `TASKS.md` and the rehearsal commit appears in the private GitHub repository history after push.	Pass / Not yet
GUI recovery performed	In GitHub Desktop History, learner reverted that exact rehearsal commit; a later recovery commit appears; the temporary line is gone and accepted project work remains.	Pass / Not yet
Recovery push verified	The later recovery commit appears in the private GitHub repository history after push.	Pass / Not yet
Privacy-safe evidence	Evidence remains local and contains no unredacted identifying details or credentials.	Pass / Not yet

Decision

- **Pass:** all 13 criteria are Pass.
- **Not yet:** any criterion is Not yet. Name the missing criterion and repeat the needed step. A local-only commit or browser-only simulation does not substitute for the required private GitHub push or local repository access.

Evidence worksheet

Complete this after the challenge. Keep evidence locally. Do not make the repository public or submit it. Use redacted observations or screenshots; do not include account names, repository URLs, email addresses, access tokens, or other identifying details.

Same local repository and fresh Codex chat

- Workspace used (own completed repository or worked example):
- Same local checkout opened in VS Code and GitHub Desktop (yes/no):
- Fresh local Codex chat opened in that workspace (not Cloud or worktree) (yes/no):
- Did Codex read the six files directly from the local workspace, without attachments or pasted contents (yes/no):
- Filenames Codex identified as actually read:
- Current state Codex summarized before the change (budget, attendance, superseded choice, unknowns, next task):
- Did you check that read-back against the actual files (yes/no):

Local edit and diff

- Files Codex actually changed in the local repository:
- Did the local edits set 30 as current in `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, and `TASKS.md` (yes/no):
- Is 40 preserved only as superseded history in `DECISIONS.md`, with no stale 40 in `KNOWLEDGE.md` (yes/no):
- Did you check `README.md` for a duplicated current amount and correct/remove it if needed (yes/no):
- Attendance remains 12 and venue remains unconfirmed (yes/no):
- No spending or messages are authorized in the record (yes/no):
- Did you inspect every actual changed-file diff in GitHub Desktop (yes/no):
- Project update commit message:
- Project update commit visible in the private GitHub repository history after push (yes/no):

Separate recovery and push

- Did the rehearsal commit change only `TASKS.md` with the exact line `Recovery rehearsal — remove me` (yes/no):
- Rehearsal commit message:
- Rehearsal commit visible in the private GitHub repository history after push (yes/no):
- Did you select that exact commit in GitHub Desktop History and use **Revert changes in commit** (yes/no):
- Recovery commit message or History observation:
- Recovery commit visible in the private GitHub repository history after push (yes/no):
- Temporary line is gone; accepted 30-unit budget and 12 attendees remain (yes/no):

Evidence kept locally

List evidence items without exposing their sensitive details. Examples: redacted screenshot of the local VS Code workspace and file list, GitHub Desktop diff, GitHub Desktop History, or the private GitHub commit history.

- Evidence item(s):
- Any criterion not completed, and why (local Codex access or missing push means Not yet):

Copy-ready Codex prompts

These prompts are for the main course path: VS Code with the official OpenAI Codex extension, opened on the same local repository checkout that GitHub Desktop manages. Start a new local Codex chat in that workspace, not a Cloud task or separate worktree, and ask it to read project files directly. Do not attach or paste file contents as a substitute for local repository access.

Codex can make local file edits within the open workspace. Name the allowed files in each request and keep the scope bounded. Review the actual changed-file list and diff in GitHub Desktop. The learner chooses whether to commit or push; do not ask Codex to do that. `AGENTS.md` is collaboration guidance, not a security boundary, so inspect what actually changed.

If the VS Code/Codex local workflow is unavailable, the separate browser fallback offers a text-only adaptation. It cannot read or edit the local repository and cannot satisfy the primary repository-awareness assessment criterion. See local Codex setup to open the correct checkout.

1. Orient

```
I am in a fresh local Codex chat in VS Code, with this repository open in the same checkout as
    GitHub Desktop. Do not use Codex Cloud or a separate worktree. Read these six files directly
    from the current workspace: README.md, AGENTS.md, GOALS.md, KNOWLEDGE.md, DECISIONS.md, and
    TASKS.md. First list the filenames you actually read from the workspace. Then explain what each
    file is for and summarize the current state, confirmed facts, assumptions, current decisions,
    superseded choices, and next action. Do not edit files. Do not infer missing information or
    claim access to files you did not read.
```

2. Direct

```
Read AGENTS.md, GOALS.md, KNOWLEDGE.md, DECISIONS.md, and TASKS.md directly from this workspace.
    Help me plan a simple 60-minute agenda for a fictional community skill-share with 12 attendees.
    Give everyone a chance to take part, use low-cost activities, and label assumptions. Do not
    change the attendance target, choose a venue, contact anyone, spend money, or claim the agenda
    is approved. Make one local edit only: add a concrete agenda-drafting action to the “Other
    tasks” section of TASKS.md. Do not edit any other file. Do not commit or push. Tell me exactly
    what you changed.
```

3. Ground

```
Read KNOWLEDGE.md and DECISIONS.md directly from this local workspace. This is a fictional
    skill-share for 12 attendees. A community room or a host's home may be possible, but neither is
    confirmed. Do not invent a date, availability, attendee names, or booking. Update KNOWLEDGE.md
    only to keep confirmed details, assumptions, unknowns, and boundaries distinct. Leave
    DECISIONS.md unchanged. Tell me what you changed and which file you read. Do not commit or push.
```

4. Decide

```
Read GOALS.md, KNOWLEDGE.md, DECISIONS.md, and TASKS.md directly from this local workspace. A new
    fictional requirement reduces the planning budget from 60 to 40 units. Keep attendance at 12.
    Make local edits only to those four named files: set 40 as the current limit in all four;
    preserve 60 only as superseded history in DECISIONS.md with the reason; and keep the next task
    consistent. Do not invent a reason beyond the stated budget change. Keep the venue unconfirmed
    and do not authorize spending, messages, bookings, or posts. Do not edit other files. Do not
    commit or push. Summarize the edits by filename.
```

For the final challenge, use the same local workflow with the budget change set to 40 → 30. Read all six files directly, inspect for duplicated current budget values, and edit only `GOALS.md`, `KNOWLEDGE.md`, `DECISIONS.md`, and `TASKS.md`, plus `README.md` only if it repeats a current budget that would become stale. Keep 40 only as superseded decision history. The full instructions are in final challenge.

5. Inspect a diff

```
Do not edit or commit anything. Read the current local versions of the files changed for this task
    and check whether they are consistent with this requirement: [describe the requested change].
    Identify unsupported claims, inconsistent current values, changes beyond the named scope, and
    boundary changes. Do not claim that you inspected a Git diff unless you can actually access it.
    I will inspect the actual changed-file list and diff in GitHub Desktop before deciding whether
    to commit or push. If you cannot see a diff, say that clearly and review only the current
    workspace files you read.
```

Then inspect the real diff yourself in GitHub Desktop's **Changes** view. Check that every changed file is in scope and that its before/after content matches the request. The assistant's summary is not a substitute for that check.

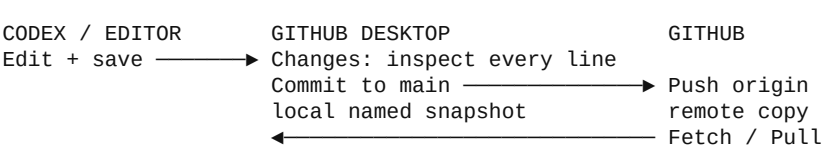
6. Resume in a fresh session

```
This is a fresh local Codex chat in VS Code, in the same checkout GitHub Desktop manages. Do not use
    Codex Cloud, a separate worktree, a previous conversation, or assumed memory. Read README.md,
    AGENTS.md, GOALS.md, KNOWLEDGE.md, DECISIONS.md, and TASKS.md directly from the local workspace
    now open. List the filenames you actually read and summarize the current state, confirmed facts,
    assumptions, open questions, current decisions, superseded choices, and next action, naming the
    file that supports each point. Check whether the files agree. Do not edit files yet. If a file
    is unavailable in this workspace, tell me which one and stop before guessing.
```

After checking that read-back against the files, ask for one bounded task and name the files Codex may edit. Start a separate fresh chat in the same checkout when practicing a new-session handoff.

Git you can see

Git keeps named snapshots on your computer. GitHub holds a remote copy. GitHub Desktop connects them. Saving a file, committing and pushing are three separate actions.



Fetch checks the remote; pull brings remote commits into your local branch. Before working on another computer, push here, then fetch and pull there. A commit alone is local.

For a visual overview, see the three workspace maps.

Inspect a change

1. Let Codex finish its authorized local edit, or save a manual edit, in the cloned learner folder.
2. Select that repository in Desktop. Open **Changes**, then select the filename.
3. Read the removed and added lines. Color is a convenience: minus means removed, plus means added. Check file names and text, not color alone.
4. Ask: does it meet my request, preserve constraints and change only intended files?
5. Select only the files belonging to this change. Type a concrete summary, such as `Reduce event budget to 40`, then **Commit to main**. If your initial branch has another name, the button uses that name.
6. **Push origin**. Open **Repository → View on GitHub** and confirm the latest commit and file text match. UI labels may vary slightly by version.

```
- Budget ceiling: 60 units.
+ Budget ceiling: 40 units.
  Attendance target: 12 people.
```

The changed budget is visible. The unchanged attendance line proves a separate constraint survived. A diff is evidence of changed text, not evidence that a real venue is available.

Case A: an unwanted edit has NOT been committed

Only practise on a harmless extra line you just added to TASKS.md. Save it and inspect the diff. Right-click that file under **Changes** and choose **Discard changes**. Read the confirmation: this discards **all uncommitted edits in that file**. Proceed only when the file contains only the drill edit. Reopen it and confirm the line vanished. If it contains wanted work, cancel and edit out just the unwanted line in your text editor instead.

Untracked new files behave differently and may be removed by discard; do not use them for this drill. Discard is not a backup. Commit accepted work before recovery practice.

Case B: an unwanted change HAS been committed

1. First commit and push your accepted project changes. Confirm **Changes** is empty.
2. Add exactly one harmless unchecked item to TASKS.md: `Recovery rehearsal — remove me`. Save, inspect and commit **only that line** with summary `Add recovery rehearsal`.
3. Open **History**. Select that exact commit and inspect its file changes. It must contain only the rehearsal item.
4. Right-click the commit → **Revert changes in commit**. Desktop creates a new commit that reverses that commit. It preserves earlier history.
5. Check TASKS.md: the rehearsal line is gone, while the accepted budget and 12-person goal remain. Inspect the new history entry, then push and verify on GitHub.

Revert reverses everything in the selected commit. Do not select a mixed commit containing accepted work. Later edits can cause conflicts: stop if a conflict appears and use the troubleshooting path below.

Recover earlier text without reversing a whole commit

In **History**, select the earlier commit affecting your file and inspect the old text/diff. Copy the desired earlier passage into your editor (or use GitHub's commit file view to read the full earlier file). Review it against current goals, save, inspect and commit it as a new change. This preserves newer work and a visible explanation. No branch reset or force push is needed.

Pulling a remote change

With a clean local **Changes** panel, click **Fetch origin**. If Desktop shows **Pull origin**, click it. Open the changed file and inspect History. If no one changed the remote, no pull is needed; fetch can correctly report nothing new. Never demonstrate synchronization by editing both copies at once.

If something goes wrong

- No changes shown: save the file, confirm the folder and selected repository are the same.
- Push rejected: verify sign-in and repository access; fetch, then pull if offered. If conflicts appear, stop and ask for help with the exact files; keep a copy of uncommitted text outside the repository first.
- Conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`): do not commit them. Resolve with guidance in your editor/Desktop, inspect the final text, then continue.
- Revert unavailable: finish or preserve existing uncommitted work first, and verify you selected a history commit.
- AI says “pushed”: check GitHub yourself. In this course you perform the push in Desktop; an assistant's claim is not verification.
- Secret included: do not assume a later deletion removes it from history. Stop sharing, revoke the exposed credential through its provider, then seek help removing sensitive history.

Optional command reference

These are optional equivalents, not prerequisites. Run inside the correct repository. Replace `COMMIT_ID` with the selected actual identifier.

```
git status
git diff
git add GOALS.md KNOWLEDGE.md DECISIONS.md TASKS.md
git commit -m "Reduce event budget to 40"
git push
git fetch
git pull --ff-only
git log --oneline
git show COMMIT_ID
```

For a separate, already-reviewed rehearsal commit only: `git revert COMMIT_ID` creates a reversal commit. `git restore -- TASKS.md` discards all uncommitted changes in that tracked file; do not run it when the file contains wanted edits. Avoid reset/force-push in this course.

Reference sheet · Course

The Living Repository — desk reference

The loop: Read → request → edit → inspect → commit → push → resume.

File	Question it answers
README.md	Where am I and how do I start?
AGENTS.md	How should an assistant collaborate here?
GOALS.md	What outcome and constraints matter?
KNOWLEDGE.md	What is known, assumed or still unknown?
DECISIONS.md	What was chosen, why, and what did it replace?
TASKS.md	What happens next and what is blocked?

Fresh-session prompt: “Read the six workspace files from this repository. Summarize the current goal, constraints, latest decision and next action. Separate facts from assumptions. Do not edit yet.” Use Codex in the same local folder as GitHub Desktop; verify its read-back against the files.

Change prompt: “Propose the smallest changes for [request]. Preserve [constraints]. List affected files. Wait while I review.” Then authorize edits to the named local files. Review the actual diff in GitHub Desktop.

Before commit: correct folder? saved files? read every changed line? constraints preserved? facts supported? only intended files? no private data?

Term	Meaning
Repository	Folder and its version history
Diff	Lines removed and added
Commit	Named local snapshot
Push	Send local commits to GitHub
Fetch	Check remote history
Pull	Bring remote commits into local branch
Discard	Remove uncommitted edits; may lose wanted work
Revert	New commit reversing a selected old commit

Recover: first identify whether the change was committed. For discard, verify the file contains only unwanted work. For revert, choose a single-purpose commit and inspect its entire diff. Check the final state and push the recovery commit.

Stop if: unexpected files change, a conflict appears, a tool claims access it lacks, or confidential information would be shared. Preserve your work and ask a focused question.

Optional commands: `git status`, `git diff`, `git log --oneline`, `git push`, `git fetch`, `git pull --ff-only`. GUI instructions and command cautions: Git guide.

Course pack and repository setup

This release is a local, publication-ready package. It has not been published as a public GitHub course or template. No live template URL is implied.

Downloadable assets

The delivered distribution folder contains:

- `the-living-repository-course-pack.zip`: complete course, guide PDF, reference PDF and the two repository ZIPs below.
- `starter-workspace.zip`: only the blank six-file learner workspace and .gitignore; no Git history.
- `worked-example-repository.zip`: an independent Git repository with real commits, lesson tags and no remote configured. Includes hidden `.git` metadata.
- `course-guide.pdf`: offline introduction, setup and six lessons plus assessment and Git references.
- `reference-sheet.pdf`: printable quick reference.

Inside the course-pack ZIP, PDFs and repository ZIPs are in `downloads/`. Source Markdown remains the editable teaching material. See START-HERE for the beginner setup.

Make your private learner repository

Use GitHub's **New repository**, select **Private**, name it `my-ai-workspace`, and leave README, license and .gitignore initialization unchecked. Create it, then choose to clone it in GitHub Desktop (or Desktop **File → Clone repository** and select the new repository). Copy the contents of `starter/` or extract `starter-workspace.zip` into that local clone. Copy the files, not an enclosing extra folder; include `.gitignore`. Do not copy `.git` from another repository. Inspect, make your first commit and push. Your course reference stays separate.

Open the reference history

Extract the worked-example ZIP to a separate folder. In Desktop choose **File → Add local repository** and select the folder containing its `.git`. Inspect **History**; the commits are an authored teaching demonstration, not a record of a real event or a novice pilot. The tags identify stages. The archive retains hidden metadata even if your file manager hides it. Do not copy this `.git` over your learner repository. To practise assessment using the reference, make a separate extracted copy first. To push that copy, use Desktop's **Publish repository**, choose a unique name, and keep it private.

Maintainer publication checklist

The course source has the requested standalone layout. After owner approval, create a dedicated public course repository and upload only this course package, never the enclosing private assistant workspace. Create a separate starter-only repository, enable its **Template repository** setting, and replace the manual setup reference with its actual URL after verifying private creation. A template created from the whole course would include the whole course, not just `starter/`. Publishing the worked example separately requires preserving its `.git` history, not only uploading its current files. Publication is outside this build.

Offline limits

You can read, edit, inspect and commit locally without internet once software is installed. AI services and pushing/pulling GitHub require connectivity. PDFs are reading companions; copy prompts from Markdown when possible. Timing and beginner usability require a live pilot.

Optional browser adaptation

If you cannot use a repository-aware assistant, you can still practise maintaining explicit project state. This adaptation teaches the file and Git concepts, but does not demonstrate the primary course's local repository read/edit competency.

1. Attach or paste the current six workspace files into a suitable assistant, clearly labeling each filename. A local path alone gives a text assistant no access.
2. Ask for a read-back of the current facts, unknowns and next action. Compare it with your files.
3. Request the smallest change and complete replacement text for only the authorized files. Review it, then copy accepted text into those files in your local editor and save.
4. Inspect the saved-file diff in GitHub Desktop. Commit, push and verify as the lessons describe. The assistant's reply does not save, commit or push your local files.
5. For a fresh session, supply the newly saved files again. Do not reuse stale attachments.

Record this route as adapted practice in your evidence. To complete the main assessment, return to Codex local setup and demonstrate actual repository reads and a bounded file edit as well as the Git tasks. If tools are unavailable, mark the affected criteria **Not yet**.